CHAPTER 5

DATA TYPES AND OPERATORS

CHAPTER ESSENCE AT A GLANCE:
Data types define memory allocation and permissible domain values, while expressive operators evaluate arithmetic, relational, and boolean logic to drive programmatic computation.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Fundamental Data Type Architecture:** Primitive types (char, int, float, double, bool) establish direct mappings to hardware memory words, defining numerical precision, byte width, and valid value ranges.

• **Type Modifiers and Memory Width Adjustment:** Modifiers (signed, unsigned, short, long) alter the bit-level interpretation and storage capacity of fundamental types to optimize memory efficiency.

• **Operator Hierarchy and Associativity:** Operators are grouped into arithmetic, relational, logical, bitwise, and assignment categories, governed by rigid precedence rules that determine algebraic evaluation order.

• **Type Conversion and Coercion Dynamics:** Expressions evaluate through automatic implicit type promotion to prevent precision loss, or through explicit programmer-specified type casting.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Storage Allocation} = \{\text{char: 1B}, \text{int: 4B}, \text{float: 4B}, \text{double: 8B}, \text{bool: 1B}\}$$

Fundamental Data Type Memory Allocations

$$\text{Range}_{\text{signed}} = [-2^{n-1}, 2^{n-1}-1], \quad \text{Range}_{\text{unsigned}} = [0, 2^{n}-1]$$

Signed vs Unsigned Integer Numeric Dynamic Ranges

$$\text{Ternary Result} = (\text{Condition}) \ ? \ (\text{Expression}_1) \ : \ (\text{Expression}_2)$$

Conditional Ternary Operator Evaluation

$$\text{Promotion Rank} = \textbf{char} \rightarrow \textbf{int} \rightarrow \textbf{unsigned} \rightarrow \textbf{long} \rightarrow \textbf{float} \rightarrow \textbf{double} \rightarrow \textbf{longdouble}$$

Implicit Type Promotion Hierarchy Spectrum

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Integer division (7 divided by 2) truncates the fractional part yielding 3, not 3.5; to retain fractions at least one operand must be a floating-point type.

• **Alert:** The single equals sign = is the assignment operator, whereas the double equals == is the relational equality comparison operator.

• **Alert:** The modulus operator % strictly requires integer operands and triggers a compiler error if applied to floating-point numbers.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Distinguish between fundamental data types and user-defined data types in C++. **[Score: 2 Marks]**

Answer & Key Valuation Steps:

• Fundamental Data Types: Built-in atomic types predefined by the C++ language compiler (e.g. int, char, float, double, bool) mapped directly to machine registers. **[1 Mark]**

• User-Defined Data Types: Custom composite types created by programmers using fundamental types to model complex entities (e.g. struct, union, class, enum). **[1 Mark]**

2. What is the role of data type modifiers in C++? Name four modifiers. **[Score: 2 Marks]**

Answer & Key Valuation Steps:

• Role of Modifiers: Keywords applied to fundamental integer and character types to alter their storage size, sign interpretation, and numerical ranges. **[1 Mark]**

• Four Modifiers: signed, unsigned, short, and long. **[1 Mark]**

3. Explain the conditional operator `?:` with syntax. Rewrite an `if-else` statement using the conditional operator. **[Score: 3 Marks]**

Answer & Key Valuation Steps:

• Syntax and Evaluation: Condition ? Expression1 : Expression2. If the Condition evaluates to true, Expression1 is evaluated; otherwise Expression2 is evaluated. **[1.5 Marks]**

• Code Replacement: Given: if(a > b) max = a; else max = b; can be written concisely as: max = (a > b) ? a : b; **[1.5 Marks]**

4. Differentiate between implicit type conversion and explicit type casting with code examples. **[Score: 3 Marks]**

Answer & Key Valuation Steps:

• Implicit Type Conversion (Type Promotion): Automatic conversion performed by the compiler when evaluating mixed-mode expressions, upgrading lower types to higher types (e.g. float x = 5 + 2.5; int 5 is promoted to float). **[1.5 Marks]**

• Explicit Type Casting: Manual conversion forced by the programmer using cast notation (e.g. float avg = (float)total divided by count; converts total to float before division). **[1.5 Marks]**

5. Evaluate the following expressions given `int a = 7, b = 2;`: (a) `a divided by b`, (b) `a % b`, and (c) `(float)a divided by b`. **[Score: 3 Marks]**

Answer & Key Valuation Steps:

• (a) a divided by b: Integer division of 7 by 2 truncates fractional portion yielding integer result 3. **[1 Mark]**

• (b) a % b: Modulus remainder operator gives the integer remainder of 7 divided by 2, which is 1. **[1 Mark]**

• (c) (float)a divided by b: Explicit cast promotes a to 7.0f, causing floating-point division 7.0 divided by 2 to yield 3.5. **[1 Mark]**

6. Explain the difference between prefix increment `++x` and postfix increment `x++` with illustrative code outputs. **[Score: 3 Marks]**

Answer & Key Valuation Steps:

• Prefix Increment (++x): Increments the variable value first, then delivers the updated value for expression evaluation (e.g. if x=5, y = ++x gives x=6, y=6). **[1.5 Marks]**

• Postfix Increment (x++): Uses the current variable value in expression evaluation first, then increments the variable afterwards (e.g. if x=5, y = x++ gives y=5, x=6). **[1.5 Marks]**

7. Write a complete C++ program to input principal, rate of interest, and time period, and compute simple interest. [Score: 4 Marks]

Answer & Key Valuation Steps:

• Header and Variable Declaration: #include <iostream>\
using namespace std;\
int main() {\
 float principal, rate, time, simpleInterest; **[1 Mark]**

• Data Input: cout << "Enter Principal, Rate, Time: ";\
cin >> principal >> rate >> time; **[1 Mark]**

• Formula Computation: simpleInterest = (principal * rate * time) divided by 100.0; **[1 Mark]**

• Result Output: cout << "Simple Interest: " << simpleInterest << endl;\
return 0;\
} **[1 Mark]**